Tabla de contenidos

1. Paradigmas de programación
2. Programación Funcional
    a. Funciones Puras
    b. Consecuencias de la Programación Funcional
    c. Conclusión

Paradigmas de programación

A inicios del curso, hablamos sobre los distintos paradigmas de las programación y que cada uno de ellos nos indica un enfoque o estrategias que nos permitirán enfrentarnos a un problema.

Recordemos algunos de los paradigmas de programación que existen:

- **Procedimental**: la solución se estructura como un programa lineal. Esto es una lista de instrucciones que indican al computador qué se debe hacer con la entrada del programa en cada paso. En *Introducción a la Programación* programamos de esta manera usando Python.
- **Vectorial**: se utiliza principalmente para programas matemáticos donde hay un paralelismo implícito en los cálculos. La programación se realiza secuencialmente y el compilador se encarga de generar paralelismo en las partes donde es posible distribuir el trabajo.
- **Declarativa**: el usuario declara un problema a resolver, luego el computador determina la mejor manera de resolver el problema de manera eficiente. Por ejemplo, al consultar una base de datos usando el lenguaje SQL, donde el usuario describe de forma general una pregunta y el computador decide por si mismo cómo mover los datos para responder esa pregunta. Otro ejemplo son los lenguajes que resuelven problemas de optimización, donde se declaran todas las restricciones y función objetivo, y es el computador el encargado de decidir cómo resolver el problema.
- **Orientada a Objetos**: esto programas modelan las funcionalidades a través de interacciones entre objetos. Se utilizan los datos/atributos de los objetos y sus comportamientos para dar sentido al programa. Es lo que hemos hecho en el primer capítulo de este curso.
- **Funcional**: es programación procedimental de alto nivel. La solución del problema se estructura como un conjunto de funciones. Estas funciones reciben entradas y generan salidas. Las funciones no tienen estado, es decir, el *output* depende exclusivamente de los datos de entrada y no de otras variables externas que puedan modificar el cómputo.

Python es un lenguaje ***multiparadigma***, es decir, las soluciones pueden ser escritas de forma procedimental, orientada a objetos o funcional. Así, nuestros programas podrían ser escritos usando los diferentes enfoques de forma simultánea.

Programación Funcional

En programación funcional, una función corresponde a lo mismo que en el contexto de las matemáticas, que significa que dado el mismo valor de entrada se genera el mismo y único valor de salida. Dado lo anterior, el valor de retorno de una función depende **solamente** de los parámetros de entrada de la función.

En este paradigma todo es visto como el *output* de una función. Además, como el *output* de una función solo depende de su *input*, siempre podemos saber el valor de una variable que guarda el resultado de una función. Bajo ninguna circunstancia esa variable cambiará de valor a menos que le asignemos el *output* de otra función. Estas características otorgan claridad al código que se escribe, pues estamos seguros de que cuando se ejecuta una función no se cambian otros valores fuera de su ámbito de alcance (*scope*).

En los siguientes *notebooks* de esta semana tendremos un acercamiento a lo que es la programación funcional, en el cual aprenderemos a utilizar distintas herramientas que nos serán de utilidad para implementar soluciones que cumplan este paradigma de programación.

Funciones Puras

Como indicábamos anteriormente, en el contexto de la programación funcional, las funciones solo dependen de los parámetros de entrada (*input*) al momento de retornar un valor (*output*) y no poseen **efectos secundarios (*side effect*)**; cuando esto sucede, diremos que el código es **puro**. Como **efectos secundarios** entenderemos que es afectar a cualquier cosa que no dependa únicamente de la función.

La "pureza" mide el grado de dependencia de la función hacia el sistema o el programa, es decir, el contexto en el cual se está ejecutando dicha función:

En resumen, **una función pura retorna lo mismo para el mismo input, y no posee efectos secuntarios**.

Veamos algunos ejemplos aplicados:

```
mi_lista = [1, 2, 3]

# Función impura
def agregar_elemento(número: int) -> None:
    mi_lista.append(número)
```

En este caso, dado que la función hace uso de `mi_lista` por lo que depende de algo más que su *input*, esto hace que la función sea **impura**.

Una forma de validar lo anterior, es que si ejecutamos la función múltiples veces podremos notar que el resultado de la ejecución cambiará, pese a que no han cambiado los parámetros de entrada de la función.:

```
for i in range(1, 6):
    agregar_elemento(4)
    print(f'Ejecución {i}: {mi_lista}')
```

```
Ejecución 1: [1, 2, 3, 4]
Ejecución 2: [1, 2, 3, 4, 4]
Ejecución 3: [1, 2, 3, 4, 4, 4]
Ejecución 4: [1, 2, 3, 4, 4, 4, 4]
Ejecución 5: [1, 2, 3, 4, 4, 4, 4, 4]
```

Para evitar que la función `agregar_elemento` sea impura, deberemos asegurar que el acceso a `mi_lista` se haga por medio del *input* de la función.

```
mi_lista = [1, 2, 3]

# Función pura
def agregar_elemento(lista: list, número: int) -> list:
    copia_lista = lista.copy()
    copia_lista.append(número)

    return copia_lista


for i in range(1, 6):
    nueva_lista = agregar_elemento(mi_lista, 4)
    print(f'Ejecución {i}: {mi_lista} {nueva_lista}')
```

```
Ejecución 1: [1, 2, 3] [1, 2, 3, 4]
Ejecución 2: [1, 2, 3] [1, 2, 3, 4]
Ejecución 3: [1, 2, 3] [1, 2, 3, 4]
Ejecución 4: [1, 2, 3] [1, 2, 3, 4]
Ejecución 5: [1, 2, 3] [1, 2, 3, 4]
```

Ahora la función `agregar_elemento` solo depende de los parámetros de entrada, entonces ¿podemos decir que la función es pura?

> Sí

¿Por qué?

> Su valor de retorno depende únicamente del valor de entrada
>
> Además, al solo modificar algo dentro de su *scope*, la ejecución de la función produce no un **efecto secundario (*side effect*)**.

En resumen, una función **una función pura retorna lo mismo para el mismo input, y no posee efectos secundarios**.

Dado que ahora tenemos una mayor claridad sobre lo que son las funciones puras, veamos otros casos:

```
def aumentar(x: int) -> int:
    return x + 1
```

¿La función `aumentar` cumple los criterios de una función pura?

- Si se mantiene el *input*, ¿siempre entrega el mismo *output*? Sí
- ¿Posee efectos secundarios? No

Entonces, `aumentar` es una función pura.

```
items = ['a', 'b', 'c']

def agregar_item(item: str) -> str:
    items.append(item)
    return item
```

¿La función `agregar_item` cumple los criterios de una función pura?

- Si se mantiene el *input*, ¿siempre entrega el mismo *output*? Sí, ya que retorna el mismo valor que se recibió
- ¿Posee efectos secundarios? Sí, ya que cambia el contenido de las lista `items`

Entonces, `agregar_item` no es una función pura.

```
from random import random

def fraccionar_random(x: float) -> float:
    return x * random()
```

¿La función `fraccionar_random` cumple los criterios de una función pura?

- Si se mantiene el *input*, ¿siempre entrega el mismo *output*? No, al poseer un valor aleatorio
- ¿Posee efectos secundarios? Sí, al llamar `random`, se altera el siguiente valor a llamar

Entonces, `fraccionar_random` no es una función pura.

```
from random import random, seed

def fraccionar_random2(x: float) -> float:
    seed(x)
    return x * random()
```

Nota: `seed()` hace que el siguiente valor obtenido de `random` sea siempre el mismo si es que se usó el mismo parámetro para `seed()`.

¿La función `fraccionar_random2` cumple los criterios de una función pura?

- ☐ Si se mantiene el *input*, ¿siempre entrega el mismo *output*? Si, al depender de la *seed*
- ☐ ¿Posee efectos secundarios? Si, al modificar la *seed* de *random*, lo que modifica los valores futuros que se hagan a `random()` fuera de la función

Entonces, `fraccionar_random2` no es una función pura.

```
from random import Random

def fraccionar_random3(x: float) -> float:
    NuevoRandom = Random(x)
    return x * NuevoRandom.random()
```

Nota: `Random` recibe como parámetro una *seed*, que hace que los valores obtenidos después de cada llamado sean siempre los mismos si es que se usó la misma *seed*.

¿La función `fraccionar_random3` cumple los criterios de una función pura?

- ☐ Si se mantiene el *input*, ¿siempre entrega el mismo *output*? Si, al depender de la *seed*
- ☐ ¿Posee efectos secundarios? No, al crear un nuevo random localmente

Entonces, `fraccionar_random3` es una función pura.

```
def aumentar_con_print(x: int) -> int:
    print('Voy a retornar', x, 'aumentado en uno')
    return x + 1
```

¿La función `aumentar_con_print` cumple los criterios de una función pura?

- ☐ Si se mantiene el *input*, ¿siempre entrega el mismo *output*? Si
- ☐ ¿Posee efectos secundarios? Si, al hacer print afecta lo que muestra el programa

Entonces, `aumentar_con_print` no es una función pura.

Como puedes haber visto, los efectos secundarios tienen un alcance mayor al esperado, por ejemplo, una función pura no puede crear archivos ni modificarlos, ya que eso puede alterar otros elementos del programa.

Consecuencias de la Programación Funcional

En base a todo lo que hemos visto, ¿cuáles serian las consecuencias de implementar funciones puras?

1. Ya no se modifican elementos externos a una función.
   ➡ Ahora, podemos descartar revisar funciones puras cuando tenemos que un valor o una estructura cambia inesperadamente.

2. Ahora el output solo depende del input.
   ➡ Esto permite que si el programa recuerda el input de una función pura, no necesita llamar nuevamente a la función.

3. Dada la ausencia de efectos secundarios, no podemos razonar mediante iteraciones como `for` o `while`.
   ➡ Nuestro código deberá funcionar enfocadas en la iteración.

Veamos un ejemplo del segundo punto:

```
def fibonacchi(n: int) -> int:
    print('buscando el número', n)
    if n < 2:
        return n
    return fibonacchi(n-1) + fibonacchi(n-2)

fibonacchi(4)
```

```
buscando el número 4
buscando el número 3
buscando el número 2
buscando el número 1
buscando el número 0
buscando el número 1
buscando el número 2
buscando el número 1
buscando el número 0

3
```

Como puedes ver, hay varias funciones repetidas, y sin contar los prints que están para mostrar cómo corre el código, solo se desea el output a partir del input. Podemos aprovechar de decirle a Python que no corra nuevamente la función con el decorador `@cache`, y que asuma que el output se mantiene dado el mismo input:

```
from functools import cache

@cache
def fibonacchi_cached(n: int) -> int:
    print('buscando el número', n)
    if n < 2:
        return n
    return fibonacchi_cached(n-1) + fibonacchi_cached(n-2)

fibonacchi_cached(4)
```

```
buscando el número 4
buscando el número 3
buscando el número 2
buscando el número 1
buscando el número 0

3
```

Como puedes ver no se imprime tantas veces como antes, y esto es porque el decorador `@cache` funciona con las **funciones puras**, y el hacer print hace que la función no sea pura. Finalmente hacemos que la función sea pura, y aumentamos el número con el que llamamos a la función para aprovechar el decorador:

```
@cache
def fibonacchi_cached(n: int) -> int:
    if n < 2:
        return n
    return fibonacchi_cached(n-1) + fibonacchi_cached(n-2)

fibonacchi_cached(987)
```

```
834287860950102330394528934511688853588568224235172913310185517257559730929613976814325232093350780830370820498426132933696528884698672040728690265120350480781601704549159152139794752039092743642581937298 58
```

Además, el que nuestras funciones sean puras también tiene consecuencias a la hora de plantear un código:

1. Ya no pensaremos en estructuras de control (`for`/`while`), ni almacenamiento.
2. El razonamiento para programar se basará en pensar **flujos de transformación de datos (*data flow*)**.
3. Cada función recibe un estado particular en el que se encuentra un dato. A partir de esto, la función lo transforma y lo retorna.

Conclusión

La correcta implementación de una solución funcional, tiene grandes efectos a la hora de plantear una solución que cumpla con todo lo visto anteriormente.

Para efectos de facilitar el aprendizaje de este curso, el material entregado en los próximos *notebooks* puede presentar *prints* dentro de funciones que debiesen ser puras.